TECHNICAL PRESENTATION REPORT

UNIT 2: DATA PREPROCESSING AND DATA MANIPULATION USING PYTHON

Submitted By

Jeya Harshini R (192424051)

Course

DSA0405 – Fundamentals of Data Science

---

1. INTRODUCTION

Data has become one of the most valuable resources in today's digital world. Organizations generate large volumes of raw data from various sources such as online transactions, sensors, social media, healthcare systems, educational institutions, and business operations. However, this raw data is often incomplete, inconsistent, duplicated, or contains missing values, making it unsuitable for direct analysis. Therefore, data preprocessing is an essential step in the data science pipeline.

Data preprocessing refers to the process of cleaning, organizing, and transforming raw data into a structured format before performing data analysis or machine learning. Proper preprocessing improves the quality of data, eliminates inconsistencies, and ensures that the analytical results are accurate and reliable.

Python has become one of the most widely used programming languages for data science because of its simple syntax, flexibility, and extensive ecosystem of libraries. Libraries such as NumPy, SciPy, Pandas, Scikit-Learn, and Matplotlib provide efficient tools for numerical computation, data manipulation, machine learning, and visualization.

This report explains the concepts of data preprocessing and demonstrates common data manipulation operations using Python and a retail sales dataset.

---

2. PYTHON AND ITS LIBRARIES

Python is a high-level, interpreted programming language that is easy to learn and widely used for data science applications. Its readability and extensive library support make it an ideal choice for handling large datasets and performing complex analytical tasks.

Several libraries are commonly used during data preprocessing:

- **NumPy** provides efficient numerical computations and array operations.
- **SciPy** offers scientific and mathematical computing functions.
- **Pandas** is used for reading, cleaning, organizing, filtering, and analyzing structured data.
- **Scikit-Learn** provides machine learning algorithms and preprocessing techniques.
- **Matplotlib** is used for creating graphs and visual representations of data.

Google Colab was used as the Integrated Development Environment (IDE) for implementing the Python programs because it allows code execution directly through a web browser without requiring software installation.

---

3. DATA PREPROCESSING

Data preprocessing is the process of converting raw data into a clean and meaningful format suitable for analysis. Since real-world datasets frequently contain missing values, duplicate records, incorrect entries, and inconsistent formatting, preprocessing becomes an important step before performing any analysis.

The main objectives of data preprocessing include:

- Improving data quality
- Removing duplicate records
- Handling missing values
- Reducing inconsistencies
- Preparing data for visualization and machine learning

Effective preprocessing increases the reliability of analytical results and helps generate better business insights.

---

4. DATA MANIPULATION USING PANDAS

Data manipulation involves organizing and modifying data so that it becomes easier to analyze. In this presentation, the Pandas library was used to perform different preprocessing operations on a retail sales dataset.

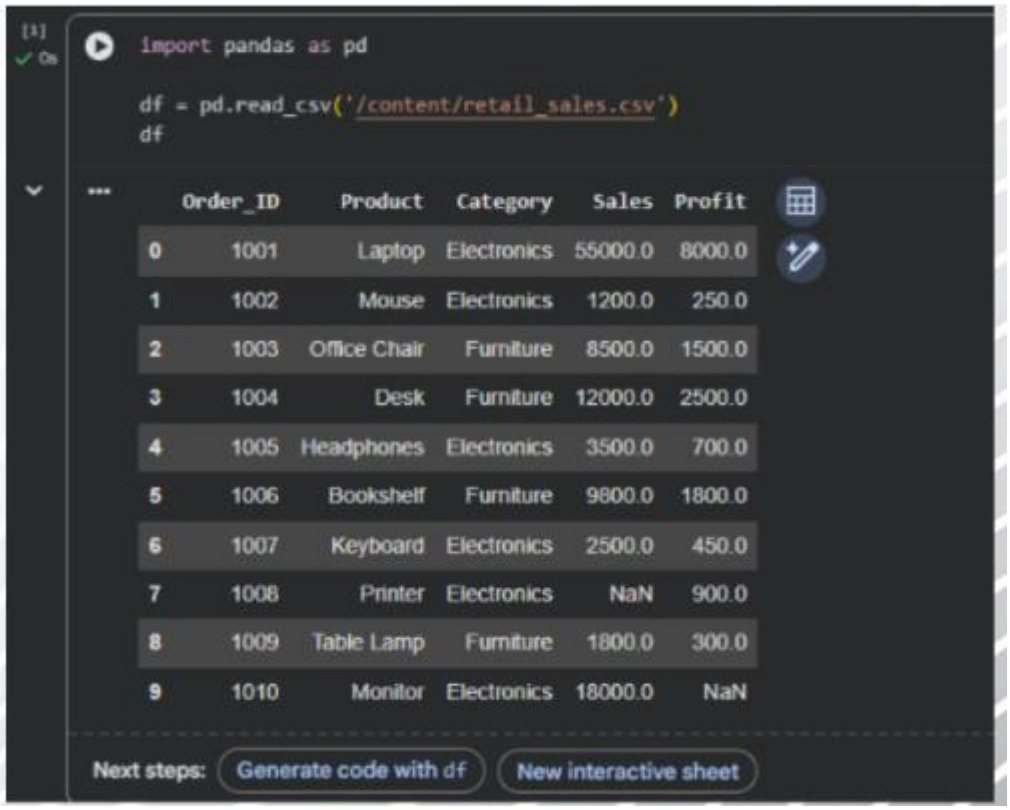
The following operations were demonstrated:

Reading Data

The retail sales dataset was imported into Python using the read_csv() function. This converts the CSV file into a Pandas DataFrame, making it easier to process.

Data Selection

Only the required columns such as Product and Sales were selected for analysis. Selecting relevant data reduces unnecessary information and improves processing efficiency.



```
[1]
✓ 0s
import pandas as pd

df = pd.read_csv('/content/retail_sales.csv')
df
```

	Order_ID	Product	Category	Sales	Profit
0	1001	Laptop	Electronics	55000.0	8000.0
1	1002	Mouse	Electronics	1200.0	250.0
2	1003	Office Chair	Furniture	8500.0	1500.0
3	1004	Desk	Furniture	12000.0	2500.0
4	1005	Headphones	Electronics	3500.0	700.0
5	1006	Bookshelf	Furniture	9800.0	1800.0
6	1007	Keyboard	Electronics	2500.0	450.0
7	1008	Printer	Electronics	NaN	900.0
8	1009	Table Lamp	Furniture	1800.0	300.0
9	1010	Monitor	Electronics	18000.0	NaN

Next steps: Generate code with df | New interactive sheet

```
[2]
✓ 0s
df[['Product','Sales']]
```

	Product	Sales
0	Laptop	55000.0
1	Mouse	1200.0
2	Office Chair	8500.0
3	Desk	12000.0
4	Headphones	3500.0
5	Bookshelf	9800.0
6	Keyboard	2500.0
7	Printer	NaN
8	Table Lamp	1800.0
9	Monitor	18000.0

Filtering Missing Data

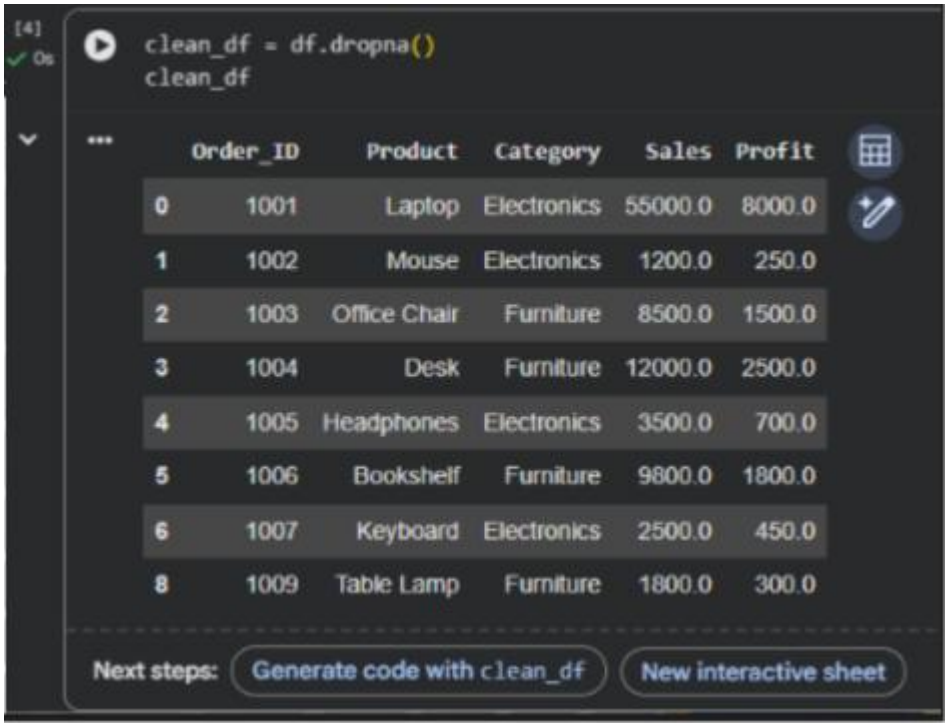
Missing values were identified using the isnull() function and removed using the dropna() function. This improved the quality and reliability of the dataset.

```
[3]
✓ 0s
df.isnull().sum()
```

	0
Order_ID	0
Product	0
Category	0
Sales	1
Profit	1

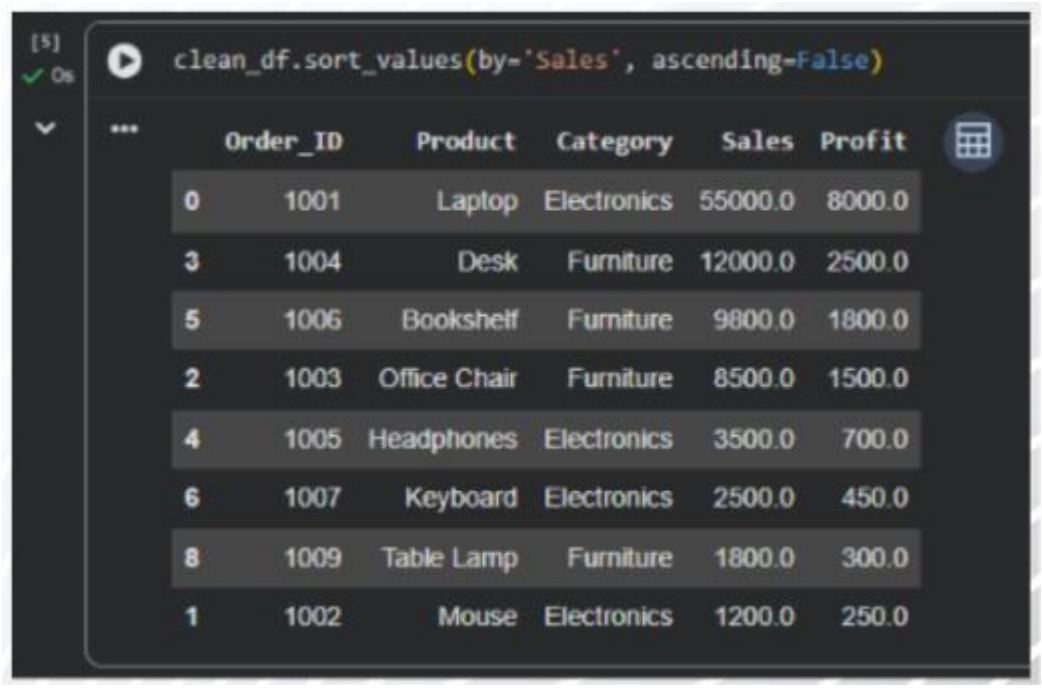
dtype: int64

[4] ✓ 0s



Sorting

The records were sorted in descending order of sales to identify the highest-selling products.



Grouping

The groupby() function was used to group products based on their category and calculate total sales for each category.

```
[6]
✓ 0s
clean_df.groupby('Category')['Sales'].sum()
```

```
...
                Sales
Category
Electronics   62200.0
Furniture     32100.0

dtype: float64
```

Ranking

Products were ranked according to their sales values. Ranking helps compare product performance and identify top-selling products.

```
[7]
✓ 0s
clean_df['Rank'] = clean_df['Sales'].rank(ascending=False)
clean_df[['Product','Sales','Rank']]
```

```
...
/tmp/ipykernel_1018/1059701776.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/st
  clean_df['Rank'] = clean_df['Sales'].rank(ascending=False)
```

	Product	Sales	Rank
0	Laptop	55000.0	1.0
1	Mouse	1200.0	8.0
2	Office Chair	8500.0	4.0
3	Desk	12000.0	2.0
4	Headphones	3500.0	5.0
5	Bookshelf	9800.0	3.0
6	Keyboard	2500.0	6.0
8	Table Lamp	1800.0	7.0

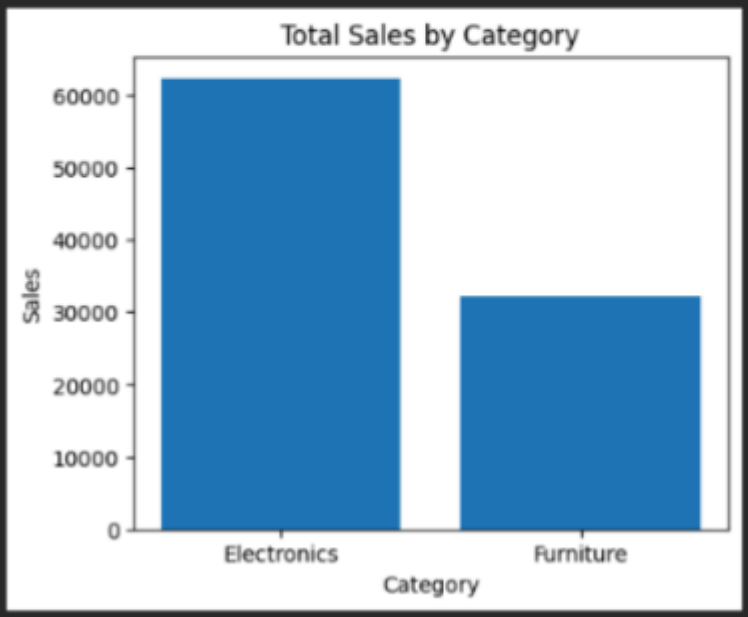
Plotting

A bar chart was created using Matplotlib to visualize the total sales of each product category. Visualization makes trends and comparisons easier to understand.

```
import matplotlib.pyplot as plt

sales = clean_df.groupby('Category')['Sales'].sum()

plt.figure(figsize=(5,4))
plt.bar(sales.index, sales.values)
plt.title("Total Sales by Category")
plt.xlabel("Category")
plt.ylabel("Sales")
plt.show()
```



---

5. PRACTICAL IMPLEMENTATION

A retail sales dataset was used to demonstrate all preprocessing operations. The dataset contained information such as Order ID, Product Name, Category, Sales, and Profit.

The following workflow was implemented:

1. Import the dataset using Pandas.
2. Display the dataset for verification.
3. Select the required columns.
4. Detect missing values.
5. Remove incomplete records.
6. Sort the cleaned dataset.
7. Group data based on product category.

8. Rank products according to sales.
9. Visualize the processed data using a bar chart.

The practical implementation was performed using Google Colab, and screenshots of the program execution were included in the presentation.

---

6. APPLICATIONS

Data preprocessing and data manipulation are widely used in many domains, including:

- Retail sales analysis
- Banking and financial analytics
- Healthcare data management
- Customer behavior analysis
- Fraud detection
- Educational data analysis
- Business intelligence
- Machine learning model preparation

These techniques improve data quality and support accurate decision-making across industries.

---

7. CONCLUSION

Data preprocessing is one of the most important stages of data science because the quality of analysis depends on the quality of the input data. Python provides a simple yet powerful environment for performing preprocessing tasks through libraries such as Pandas and Matplotlib.

In this presentation, a retail sales dataset was used to demonstrate essential data manipulation operations including reading data, selecting columns, handling missing values, sorting, grouping, ranking, and visualization. These operations transformed raw business data into structured information suitable for analysis. Overall, Python-based preprocessing improves data accuracy, simplifies analysis, and forms the foundation for effective data analytics and machine learning applications.

REFERENCES

1. Wes McKinney, *Python for Data Analysis*, O'Reilly Media.
2. Python Software Foundation. *Python Documentation*. https://docs.python.org/
3. Pandas Documentation. https://pandas.pydata.org/docs/
4. NumPy Documentation. https://numpy.org/doc/
5. Matplotlib Documentation. https://matplotlib.org/stable/